

Traitement d'Images

Master 2 Informatique

Rudi GIOT

2026-09-01

Table of contents

Accueil	3
Approche Pédagogique	3
Environnement de Travail	3
I I. Fondamentaux	4
1 Introduction	5
2 Introduction à l'acquisition d'images	6
2.1 1. Simulation d'un capteur	6
3 Filtrage	8
II II. Cas pratiques	9
4 Segmentation	10
5 Use cases	11

Accueil

Bienvenue dans le support du cours de **Traitement d'Images**, destiné aux étudiants de 2ème Master en Informatique de l'ISIB-HE2B.

L'objectif de ce cours est de faire le pont entre les fondements théoriques de la vision par ordinateur et le déploiement d'algorithmes robustes dans un contexte industriel (contrôle qualité, vision robotique, mesure de pièces).

Approche Pédagogique

Ce support suit la philosophie du *Docs as Code*. Il ne s'agit pas d'un simple recueil de diapositives, mais d'une documentation technique vivante comprenant :

- **Le cadre théorique** : Les principes mathématiques et algorithmiques.
- **L'implémentation Python** : Du code basé sur les standards de l'industrie (`OpenCV`, `scikit-image`, `NumPy`).
- **Des Use Cases** : Des cas concrets avec des jeux de données.

Environnement de Travail

Pour les séances d'exercices et la réalisation de vos projets, vous aurez besoin de configurer votre environnement local.

Prérequis d'installation

Assurez-vous de disposer des éléments suivants sur votre machine : 1. **Python 3.10+** avec un environnement virtuel configuré (`venv` ou `conda`). 2. Les bibliothèques listées dans le `requirements.txt` du cours (fourni au premier cours). 3. **VS Code** avec l'extension Jupyter pour exécuter les notebooks fournis.

Part I

I. Fondamentaux

1 Introduction

2 Introduction à l'acquisition d'images

En milieu industriel, l'image parfaite n'existe pas. Les variations d'éclairage, les vibrations des convoyeurs et le bruit thermique des capteurs nécessitent toujours un prétraitement avant de lancer une analyse de conformité.

2.1 1. Simulation d'un capteur

Plutôt que d'utiliser une image existante, simulons une pièce mécanique percée (un disque) acquise par un capteur générant du bruit thermique.

C'est ici que la puissance de Python et des bibliothèques comme NumPy et OpenCV entre en jeu.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Création d'une image noire 200x200
image_nette = np.zeros((200, 200), dtype=np.uint8)

# 2. Dessin d'un trou percé (cercle blanc au centre)
cv2.circle(image_nette, (100, 100), 50, 255, -1)

# 3. Ajout d'un bruit thermique (typique des caméras à gain élevé)
bruit = np.random.randint(0, 70, (200, 200), dtype=np.uint8)
image_bruitee = cv2.add(image_nette, bruit)

# 4. Affichage avec Matplotlib
plt.imshow(image_bruitee, cmap='gray')
plt.axis('off')
plt.show()
```

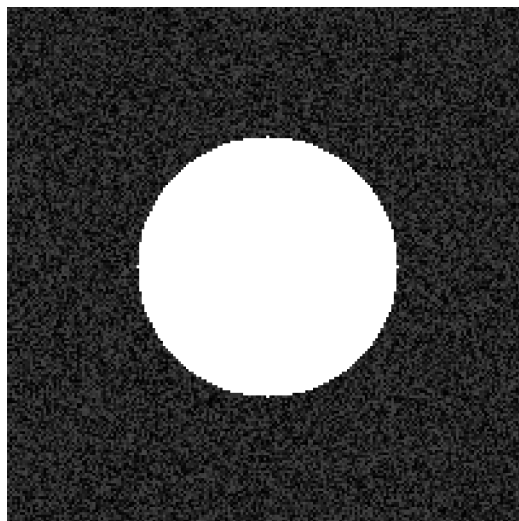


Figure 2.1: Simulation d'une acquisition caméra bruitée sur une ligne de production.

Comme le montre la Figure [2.1](#), la première étape d'un algorithme de vision consistera souvent à lisser ce signal avant d'en extraire les contours.

3 Filtrage

This is a book created from markdown and executable code.

See @knuth84 for additional discussion of literate programming.

Part II

II. Cas pratiques

4 Segmentation

This is a book created from markdown and executable code.

See @knuth84 for additional discussion of literate programming.

5 Use cases

This is a book created from markdown and executable code.

See @knuth84 for additional discussion of literate programming.